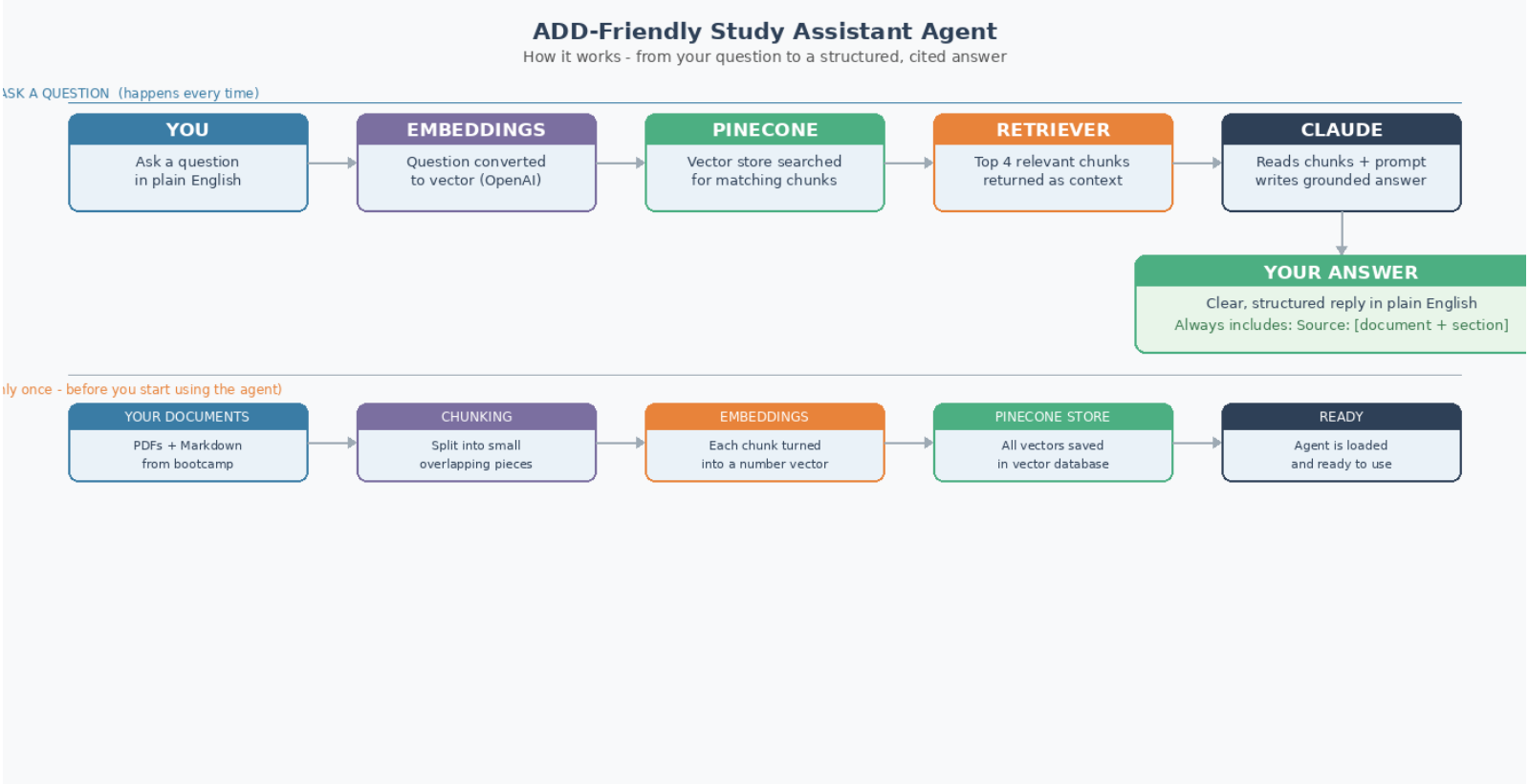
Autonomous Agent Project Plan

(ADD-Friendly) Study Assistant Agent

Dina Bosma-Buczynska



1. Use Case

I have ADD, and studying from large texts is hard. I open a document, read a few pages, get distracted, and have no idea what I absorbed. There is no easy way to ask a document a direct question and get a grounded answer with a source. I end up re-reading the same material multiple times.

This agent sits on top of my bootcamp course materials. I ask it a question in plain English, it pulls the relevant answer from my actual documents (not from an LLM's general knowledge), and tells me exactly which file it came from.

Who	Problem today	Success criteria
Students with ADD and/or in intensive programs	Search in PDFs, scattered notes, re-reading same lessons, LLM info overload	90%+ accuracy from loaded docs | 100% citation rate | under 10 sec response | plain English input works

2. Technology Stack

Component	Choice	Why / Trade-off
LLM	Anthropic Claude (claude-sonnet-4-5)	Excellent at following strict instructions which is critical for the citation rule. Considered GPT-4 but Claude is more consistent at staying within retrieved context.
Embeddings	OpenAI text-embedding-3-small	Must stay consistent with what is already in Pinecone from previous labs. Kept OpenAI for embeddings only.

Vector DB	Pinecone	Managed, free tier, already configured from previous labs. ChromaDB considered but Pinecone needs no local setup.
Framework	LangChain + LangGraph (v2)	LangChain for the MVP RAG pipeline. LangGraph added in v2 for the autonomous reasoning layer with conditional retry loops.
Interface	Gradio (prototype)	Fast to build, browser-based, no code editing needed. Deployed at gradio.live for immediate testing.

3. MVP Scope

Prototype (built) vs Full Autonomous Agent (v2)

The prototype is a RAG pipeline: one question in, one cited answer out, manually triggered. It is not autonomous yet. A true autonomous agent receives a topic, generates its own sub-questions, searches independently, retries on weak results, and synthesizes everything without being told what to do at each step. That is v2.

The key difference for the user is not technical: it is about who does the thinking. With the prototype, the student has to know what questions to ask, ask them one by one, and piece the answers together themselves. With the autonomous agent, the student types a learning goal ("help me understand LangGraph") and the agent breaks it into sub-questions, researches each one, retries if results are weak, and returns a complete structured explanation with sources. For a student with ADD, removing the cognitive overhead of question formulation is the entire point.

Prototype (done)	Full Agent v2 (planned)	Out of scope
All 15 KB docs loaded via Path.glob Batched embedding (100 chunks/call) - Pinecone namespace isolation - Source labels: [From filename] - Gradio browser interface - Fallback: not-found response tested	LangGraph: plan + research + synthesize nodes ReAct retry loop on weak retrieval Cohere reranking (Lab 3.02 pattern) Session memory for follow-up questions PDF support via pypdf	Cross-session tracking Quiz/flashcard generation Mobile app Calendar integration Auto-retraining

4. Risk Assessment

Risk	Prob	Impact	Mitigation
LLM hallucinates answer not in documents	Med	High	Prompt: only answer from context. If not found, say so.
Poor chunking retrieves irrelevant sections	High	High	Test retrieval quality before connecting LLM. Adjust chunk_size if needed.
KB summaries too thin for complex questions	High	Med	Add fallback pointing student to the source document for more detail.
API costs too high in long sessions	Med	Med	Cache frequent questions. OpenAI used only for cheap embedding calls.

5. Implementation Plan

Phase	Status	Key deliverables	Depends on
1: RAG foundation + full KB	DONE	15 docs loaded, batched embedding, Pinecone namespace, source labels, Gradio interface, fallback tested	API keys active, KB .md files ready

2: Improved retrieval	Next	Cohere reranking, PDF support, chunk size tuning	Phase 1 retrieval quality verified
3: Autonomous reasoning layer	Planned	LangGraph: plan / research / synthesize nodes, ReAct retry on weak results	Phase 2 complete
4: Deploy + document	Planned	Session memory, text input UI, user guide, tested on 5 topics without manual intervention	Phase 3 agent passing end-to-end tests

6. Success Metrics

Metric	Target	How I will measure it
Answer accuracy	90%+	20 hand-written test questions from KB docs. Correct = right info AND right source cited.
Citation rate	100%	Same 20 questions. Any answer missing a source = failure.
Response time	< 10 sec	Time each of the 20 test questions. Flag any exceeding threshold.
Autonomy (v2)	No manual steps	Given a topic, agent produces full structured explanation without any intervention between steps.

7. Resources

Solo student project.

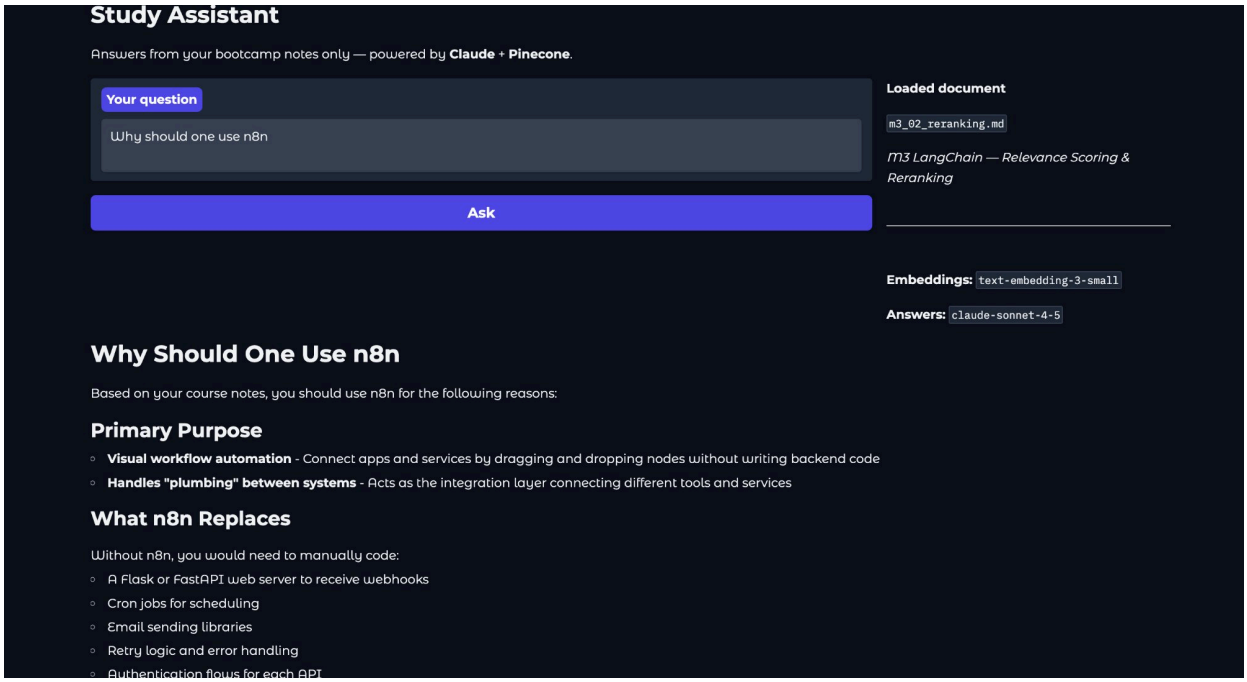
Tools used: Anthropic API (claude-sonnet-4-5, answering), OpenAI API (text-embedding-3-small, embeddings only), Pinecone free tier (1 index, 100k vectors), LangChain, Gradio, Python/Jupyter, VS Code, GitHub.
Budget: effectively zero for prototype scale. Embedding calls cost a few cents total.

No paid subscriptions needed.

8. Prototype: What Was Built and Learned

Gradio interface (live at gradio.live)

Instead of editing a question variable in a notebook, the prototype includes a Gradio browser interface. The student types a question, clicks Ask, and gets a structured cited answer. The right panel shows which document was loaded, the embedding model, and the answering model. No code editing required.



Gradio prototype interface — question: 'Why should one use n8n'

Key technical learnings

- Batching embeddings in groups of 100 is required - one large call fails above a size limit
- Pinecone namespaces isolate this project's vectors from others in the same index (important on free tier with only 5 index')
- Adding [From filename] labels to each chunk in the context means Claude cites the exact source file
- Fallback testing matters: the Flask question (not in KB) correctly returned the not-found response
- Anthropic response is at response.content[0].text, not response.choices[0].message.content like OpenAI
- time.sleep(10) after creating a Pinecone index is required - the index is not immediately usable

Reflection

Most challenging

The hardest part of this lab for me was deciding what goes in the MVP and what does not. Every feature feels important when you are the one who would use it. The framework question that helped most was: what is the smallest thing that still delivers real value? Once I answered that, the rest of the cutting was easier.

Also one must be honest about what the prototype actually is.

A RAG pipeline that answers one question is not an autonomous agent. Recognising that gap and mapping out what v2 needs to look like (LangGraph, ReAct loop, autonomous sub-question generation) was the harder and more valuable part of planning.

What I would do differently

Test the chunking strategy first, before writing any agent code. I had it as a risk in the plan but would now make it the very first thing in Phase 1 - verifying that retrieval returns relevant chunks before connecting the LLM at all. I am also aware that retrieval quality is the part most likely to go wrong. A lot of the bootcamp materials are dense and technical, and if the chunking is off the agent will just retrieve useless paragraphs. That is why I want to test retrieval before connecting the LLM, instead of just hoping it works.

Open questions

- Is LangGraph or a LangChain ReAct agent simpler to build first for the autonomous reasoning layer?
- How should the agent handle a topic where the knowledge base has almost no relevant content?
- Does Cohere reranking (already learned in Lab 3.02) improve answer quality enough to justify the extra API call?
- What is the right chunking strategy for Jupyter notebooks, which mix code, markdown, and outputs?